

Bài tiểu luận giữa kỳ

Hệ thống hội thoại dựa trên flowchart

1st Nguyễn Quang Dũng
23021497
Trường Đại học Công nghệ
Đại học Quốc gia Hà Nội
Hà Nội, Việt Nam

2nd Lê Tùng Dương
23021509
Trường Đại học Công nghệ
Đại học Quốc gia Hà Nội
Hà Nội, Việt Nam

3rd Nguyễn Kim Trung Đức
23021533
Trường Đại học Công nghệ
Đại học Quốc gia Hà Nội
Hà Nội, Việt Nam

4th Vũ Huy Đông
23021529
Trường Đại học Công nghệ
Đại học Quốc gia Hà Nội
Hà Nội, Việt Nam

5th Phạm Trung Hiếu
23021553
Trường Đại học Công nghệ
Đại học Quốc gia Hà Nội
Hà Nội, Việt Nam

Tóm tắt nội dung—Bài tiểu luận trình bày bài toán xây dựng hệ thống hội thoại dựa trên flowchart trong bối cảnh các mô hình ngôn ngữ lớn tuy mạnh về giao tiếp tự nhiên nhưng vẫn đối mặt với hiện tượng ảo giác và khó đảm bảo tuân thủ nghiêm ngặt các quy trình nghiệp vụ nhiều bước. Từ các yêu cầu về tính chính xác, minh bạch và khả năng kiểm soát trong các miền ứng dụng như hỗ trợ kỹ thuật, dịch vụ công, y tế và tài chính, nhóm phân tích vai trò của flowchart như một cơ chế ràng buộc trạng thái và điều hướng hội thoại. Trên cơ sở đó, bài viết đề xuất hướng tiếp cận kết hợp flowchart với LLM, trong đó flowchart đóng vai trò bộ định tuyến logic còn LLM đảm nhiệm giao tiếp ngôn ngữ tự nhiên và suy luận cục bộ theo trạng thái hiện tại. Cách tiếp cận này hướng tới mục tiêu vừa giữ được sự linh hoạt trong tương tác, vừa bảo đảm tính nhất quán logic, khả năng bảo trì và độ tin cậy của hệ thống hội thoại trong các bài toán nghiệp vụ có cấu trúc.

Từ khóa—hệ thống hội thoại, flowchart, mô hình ngôn ngữ lớn, điều hướng hội thoại, xử lý sự cố, trí tuệ nhân tạo

I. MÔ TẢ BÀI TOÁN

A. Bối cảnh

Sự phát triển vượt bậc của các mô hình ngôn ngữ lớn (LLM) đã mang lại những tiến bộ đáng kể cho hệ thống AI hội thoại nhờ khả năng tương tác tự nhiên và giải quyết tác vụ linh hoạt [1]. Tuy nhiên, rào cản lớn nhất khi ứng dụng các mô hình này vào các quy trình thực tế là hiện tượng ảo giác (hallucination), xảy ra khi các mô hình tạo ra các thông tin sai lệch nhưng được trình bày một cách rất thuyết phục. Đáng chú ý, các nghiên cứu gần đây đã chứng minh rằng ảo giác là một hạn chế toán học nội tại của quá trình tổng hợp thông tin, khiến việc kiểm soát hoàn toàn tính chính xác trở thành một thách thức vô cùng lớn, không thể giải quyết triệt để chỉ bằng cách mở rộng dữ liệu hay tinh chỉnh mô hình [1], [2].

Trong các lĩnh vực nghiệp vụ chuyên sâu yêu cầu độ chính xác tuyệt đối như y tế, pháp lý hay hỗ trợ kỹ thuật, việc hệ thống cung cấp thông tin sai lệch có thể gây ra những rủi ro nghiêm trọng. Nhằm tránh các rủi ro đó, các hệ thống phải

được thiết kế để tuân thủ các quy trình phức tạp, dẫn dắt người dùng qua nhiều bước ra quyết định. Những quy trình này thường được chuẩn hóa chặt chẽ dưới dạng các flowchart [3]. Do thiếu cơ chế kiểm soát cấu trúc và tính nhất quán logic, các mô hình ngôn ngữ tự do thường gặp khó khăn và dễ đi chệch hướng khi phải tuân thủ các quy trình đa bước này.

Để khắc phục nhược điểm trên, việc phát triển các hệ thống hội thoại dựa trên flowchart đang trở thành một hướng tiếp cận thiết yếu. Bằng cách kết hợp khả năng hiểu ngôn ngữ linh hoạt của các mô hình hiện đại với các ràng buộc chặt chẽ từ cấu trúc đồ thị, hệ thống có thể đảm bảo mọi bước chuyển trạng thái đều hợp lệ và minh bạch. Điều này giúp tạo ra các tác nhân hội thoại không chỉ giao tiếp lưu loát mà còn đảm bảo tính trung thực và tuân thủ tuyệt đối các quy tắc nghiệp vụ đã được định sẵn [3]–[5].

B. Định nghĩa bài toán

Bài toán hội thoại hướng flowchart (Flowchart-oriented Dialogue - FOD) được định nghĩa như sau [5]: Một flowchart $\mathcal{F}$ gồm 3 thành phần $(\mathcal{V}, \mathcal{E}, \mathcal{X})$, trong đó:

- $\mathcal{V} = \{v_1, \dots, v_{|\mathcal{V}|}\}$ là tập hợp các nút. Mỗi nút đại diện cho một câu hỏi chẩn đoán hoặc các bước thực hiện quy trình cụ thể.
- $\mathcal{E} = \{e_1, \dots, e_{|\mathcal{E}|}\}$ là tập hợp các cạnh. Mỗi cạnh $e = (v_i, v_j) \in \mathcal{E}$ xác định một bước chuyển tiếp từ nút nguồn v_i sang nút đích v_j .
- $\mathcal{X} = \{x_v | v \in \mathcal{V}\} \cup \{x_e | e \in \mathcal{E}\}$ biểu thị các thuộc tính văn bản đi kèm với các nút và cạnh. Thuộc tính của cạnh mã hóa điều kiện cần thiết để thực hiện chuyển tiếp giữa các nút.

Một cuộc hội thoại hướng flowchart $\mathcal{D}$ bao gồm T lượt tương tác luân phiên giữa người dùng và agent, được ký hiệu là $\mathcal{D} = \{(c_1^u, c_1^a), \dots, (c_T^u, c_T^a)\}$. Tại mỗi lượt t , dựa trên flowchart

$\mathcal{F}$ và lịch sử hội thoại $\mathcal{D}_t = \{(c_1^u, c_1^a), \dots, (c_{t-1}^u, c_{t-1}^a), c_t^u\}$, tác nhân cần thực hiện hai mục tiêu chính:

- 1) **Lập luận flowchart:** Agent phải xác định ngữ cảnh hội thoại hiện tại vào một nút $\hat{v}_t \in \mathcal{V}$ trong flowchart. Chuỗi nút được xác định $(\hat{v}_1, \hat{v}_2, \dots, \hat{v}_t)$ phải đảm bảo tính nhất quán logic với các bước chuyển tiếp trong flowchart:

$$(\hat{v}_k, \hat{v}_{k+1}) \in \mathcal{E}, \quad \forall k = 1, \dots, t-1 \quad (1)$$

- 2) **Tạo phản hồi:** Dựa trên $\mathcal{D}_t$ và nút đã xác định $\hat{v}_t$, agent tạo ra một câu trả lời $\hat{c}_t^a$ phù hợp với kết quả lập luận và duy trì tính mạch lạc của cuộc hội thoại. Đây là mục tiêu quan trọng nhất của bài toán.

C. Lý do cần kỹ thuật Flowchart

Phương pháp sử dụng flowchart làm nền tảng cho hệ thống hội thoại là giải pháp cần thiết để khắc phục những điểm yếu của các mô hình ngôn ngữ lớn trong môi trường nghiệp vụ khắt khe [2], [3]. Các lý do chính bao gồm:

- **Tính kiểm soát:** Flowchart đóng vai trò như một bộ khung điều hướng, ngăn LLM đi chệch khỏi quy trình do nhiễu dữ liệu hoặc câu lệnh đầu vào không nhất quán [1]. Việc tuân thủ nghiêm ngặt các bước nghiệp vụ là cực kỳ quan trọng trong xử lý sự cố, giúp tránh các kết luận sai lầm hoặc nguy cơ hỏng hóc thiết bị [3], [4].
- **Tính minh bạch:** Các mô hình LLM thường hoạt động như một hộp đen [1]. Ngược lại, flowchart cung cấp một lộ trình logic rõ ràng. Người quản trị luôn biết hệ thống đang ở node nào và lý do đưa ra phản hồi, từ đó dễ dàng thực hiện công tác hậu kiểm và kiểm soát chất lượng [3].
- **Khả năng bảo trì:** Việc thay đổi quy tắc trong LLM tinh chỉnh thường tốn kém vì phải thu thập lại dữ liệu và huấn luyện lại [6], [7]. Với flowchart, logic hội thoại được tách biệt khỏi mô hình [3]. Chuyên gia nghiệp vụ có thể trực tiếp sửa đổi sơ đồ mà không cần can thiệp mã nguồn hay huấn luyện lại hệ thống [3].
- **Độ tin cậy:** Bằng cách ràng buộc không gian phản hồi vào flowchart, hệ thống loại bỏ hiện tượng *ảo giác* về mặt quy trình [2]. Chatbot không thể tự tạo ra các giải pháp kỹ thuật không tồn tại nếu chúng không có mặt trong sơ đồ logic đã được neo giữ [3], [4].

D. Đối tượng và phạm vi

Hệ thống hội thoại dựa trên flowchart được thiết kế để giải quyết các bài toán đặc thù trong những lĩnh vực yêu cầu sự chính xác về quy trình và tính chuyên môn cao. Phạm vi ứng dụng và đối tượng phục vụ của kỹ thuật này có sự phân hóa rõ rệt so với các hệ thống LLM tổng quát.

Các lĩnh vực ứng dụng trọng tâm của các hệ thống hội thoại ứng dụng flowchart là:

- **Hỗ trợ kỹ thuật và xử lý sự cố:** Đây là lĩnh vực điển hình nhất, nơi các quy trình sửa chữa thiết bị (như ô tô, máy tính, thiết bị gia dụng) đã được chuẩn hóa thành các bước kiểm tra logic [3].
- **Dịch vụ công và hành chính:** Hướng dẫn người dùng thực hiện các thủ tục giấy tờ, phúc lợi xã hội dựa trên các văn bản quy định pháp luật phức tạp [4].

- **Y tế và tài chính:** Tư vấn sơ bộ hoặc phân loại khách hàng dựa trên các triệu chứng hoặc điều kiện tài chính cụ thể, nơi mà việc đưa ra thông tin sai lệch có rủi ro cao [1].

Các đối tượng phục vụ chính của các hệ thống hội thoại ứng dụng flowchart là:

- **Người dùng cuối:** Những người cần giải quyết một vấn đề cụ thể mà không có kiến thức chuyên môn, cần sự dẫn dắt từng bước để đi đến giải pháp [3], [4].
- **Quản trị viên nghiệp vụ:** Những người chịu trách nhiệm về nội dung và logic nhưng không có kỹ năng lập trình chuyên sâu, có thể dễ dàng quản lý hệ thống thông qua việc chỉnh sửa sơ đồ [3].

II. NGHIÊN CỨU BÀI TOÁN

A. Các nghiên cứu liên quan

1. Phương pháp sử dụng ô tô-mát hữu hạn trạng thái

Những hệ thống hội thoại thể hệ đầu sử dụng ô tô-mát hữu hạn trạng thái (Finite State Automata - FSA) để mô hình hóa luồng hội thoại [8]. Mỗi trạng thái đại diện cho một bước trong hội thoại, và các chuyển tiếp được kích hoạt bởi đầu vào của người dùng. Về mặt hình thức, một hệ thống hội thoại dựa trên FSA được định nghĩa bằng bộ năm $(Q, \Sigma, \delta, q_0, F)$, trong đó Q là tập trạng thái, Σ là bảng chữ cái đầu vào, $\delta : Q \times \Sigma \rightarrow Q$ là hàm chuyển, q_0 là trạng thái khởi đầu và F là tập trạng thái kết thúc [9].

Kopeček và Škarvada [9] đã chứng minh rằng mọi hệ thống hội thoại đều có thể được biểu diễn dưới dạng ô tô-mát hữu hạn thông qua một phép xây dựng phổ quát từ ngữ liệu hội thoại. FSA cung cấp một biểu diễn trực quan, dễ hiểu, cho phép các nhà phát triển dễ dàng theo dõi và gỡ lỗi luồng hội thoại [9]. Phương pháp của Kopeček và Škarvada [9] cho phép tự động xây dựng ô tô-mát từ kho ngữ liệu hội thoại, mở ra khả năng tự động hóa việc phát triển hệ thống.

Tuy nhiên, FSA yêu cầu dự đoán trước tất cả các đường dẫn hội thoại có thể xảy ra, điều này không khả thi đối với các miền ứng dụng phức tạp [8]. Ngoài ra, FSA truyền thống không có cơ chế để biểu diễn độ không chắc chắn trong hành vi của người dùng hoặc lỗi nhận dạng giọng nói [10].

2. Phương pháp sử dụng các mô hình xác suất

Để khắc phục tính cứng nhắc của FSA, Orozko và Torres [11] đã giới thiệu Stochastic Bi-Automata, một mô hình xác suất hai chiều có khả năng học trực tuyến (online learning). Mô hình gán xác suất cho mỗi phép chuyển trạng thái và bao gồm hai ô tô-mát song song (một cho người dùng, một cho hệ thống) [11]. Các tác giả đề xuất thuật toán ước lượng tham số cho phép cập nhật sau mỗi tương tác [11].

Đây là bước tiến quan trọng từ mô hình tất định sang xác suất, lần đầu tiên cho phép mô hình hóa độ không chắc chắn trong hành vi của người dùng [11]. Khả năng học trực tuyến giúp hệ thống thích ứng với từng người dùng cụ thể mà không cần can thiệp thủ công [11]. Cấu trúc hai chiều với hai ô tô-mát song song là một ý tưởng sáng tạo, ảnh hưởng đến nhiều công trình sau này như A-PFSBA [10].

Tuy nhiên, mô hình mới chỉ được thử nghiệm trên ngữ liệu nhỏ, chưa được đánh giá trên các flowchart phức tạp với nhiều

nhánh rẽ [11]. Ngoài ra, việc học trực tuyến đòi hỏi khởi tạo tham số ban đầu tốt, nếu khởi tạo kém có thể dẫn đến hội tụ chậm hoặc sai lệch

3. Phương pháp sử dụng mô hình ngôn ngữ lớn

Năm 2021, Raghu và cộng sự [12] đã đề xuất FLONET (FLOWchart grounded NETwork), một trong những phương pháp đầu tiên sử dụng mô hình ngôn ngữ lớn cho bài toán hội thoại dựa trên flowchart. FLONET sử dụng kết hợp LLM và kỹ thuật tạo sinh dựa trên truy xuất tăng cường (Retrieval Augmented Generation - RAG), trong đó một bộ truy xuất (retriever) tìm kiếm các đoạn hội thoại liên quan từ một kho các câu hỏi thường gặp, và một bộ sinh tạo ra phản hồi dựa trên thông tin được truy xuất [12].

Điểm đặc biệt của FLONET là cách nó biểu diễn flowchart: toàn bộ flowchart được xử lý như một túi các nút, chỉ chứa tập hợp các câu hỏi hoặc hướng dẫn trong flowchart mà hoàn toàn bỏ qua cấu trúc kết nối giữa chúng [12]. Mỗi nút được mã hóa thành vector bằng BERT, sau đó các vector này được tổng hợp lại thành một biểu diễn duy nhất cho toàn bộ flowchart [12].

FLONET là công trình tiên phong chứng minh tính khả thi của việc áp dụng học sâu vào bài toán hội thoại dựa trên flowchart, đạt các chỉ số BLEU và ROUGE cạnh tranh trên ngữ liệu FloDial [12]. Nó mở đường cho một hướng nghiên cứu mới, khác biệt hoàn toàn so với các phương pháp FSA hay mô hình xác suất trước đó [13].

FLONET có ba hạn chế lớn:

Một là mất thông tin cấu trúc. Biểu diễn túi các nút khiến FLONET không thể phân biệt được nút A có dẫn đến nút B hay nút C. Điều này dẫn đến việc sinh ra các phản hồi không nhất quán với logic của flowchart. Phân tích lỗi cho thấy khoảng 35% lỗi của FLONET đến từ việc vi phạm ràng buộc cấu trúc [13].

Hai là không hiểu được câu trả lời gián tiếp. Trong hội thoại thực tế, người dùng thường trả lời các câu hỏi có hoặc không một cách gián tiếp (ví dụ: "Xe của tôi mới mua" để trả lời cho câu hỏi "Xe của bạn có cũ không?"). FLONET không có cơ chế suy luận để xác định tính có hoặc không của các câu trả lời này, chỉ đạt độ chính xác khoảng 62% trong tác vụ này [13].

Ba là phụ thuộc vào dữ liệu huấn luyện. FLONET cần lượng lớn dữ liệu hội thoại có gán nhãn, điều này rất tốn kém và đòi hỏi chuyên gia trong lĩnh vực [12], [13].

Để khắc phục các hạn chế của FLONET, Raghu và cộng sự [13] đã đề xuất Structure-Aware FLONET (SA-FLONET) với hai cải tiến chính, nhằm giải quyết lần lượt hai hạn chế đầu tiên của FLONET.

Thứ nhất là cải tiến về ràng buộc cấu trúc (Structural constraints). Thay vì biểu diễn flowchart như một túi các nút, SA-FLONET xây dựng một ma trận kề (adjacency matrix) để biểu diễn các kết nối hợp lệ giữa các nút [13]. Trong quá trình sinh phản hồi, mô hình chỉ được phép chuyển đến các nút là láng giềng của nút hiện tại trên flowchart. Ràng buộc này được đưa trực tiếp vào hàm mất mát khi huấn luyện, buộc mô hình học được cấu trúc luồng của flowchart [13].

Thứ hai là cải tiến về suy luận ngôn ngữ tự nhiên (Natural language inference). SA-FLONET tích hợp một mô hình NLI

được huấn luyện riêng để xác định tính có hoặc không của các câu trả lời gián tiếp từ người dùng [13]. Cụ thể, mô hình NLI dự đoán mối quan hệ kéo theo giữa câu hỏi và câu trả lời của người dùng [13].

SA-FLONET đạt được những cải thiện đáng kể so với FLONET [13]. Tỷ lệ thành công trong tác vụ sinh phản hồi dựa trên flowchart được cải thiện 68%. Trong tác vụ zero-shot (trên các flowchart chưa thấy trong huấn luyện), mức cải thiện lên đến 123%. Đồng thời, độ chính xác trong việc xác định tính có hoặc không của câu trả lời gián tiếp tăng từ 62% lên 86% nhờ cơ chế NLI

Mặc dù vượt trội so với FLONET, SA-FLONET vẫn còn một hạn chế quan trọng: nó vẫn yêu cầu một lượng lớn dữ liệu hội thoại có gán nhãn để huấn luyện [13]. Điều này đặt ra bài toán về khả năng mở rộng sang các lĩnh vực mới hoặc các ngôn ngữ khác, nơi không có sẵn dữ liệu chất lượng cao [14].

B. Các ví dụ thực tế

1) Hệ thống phản hồi thoại tương tác:

a) **Giới thiệu:** Hệ thống trả lời tự động (**Interactive Voice Response, viết tắt là IVR**) là một công nghệ cho phép các máy móc tự động tương tác với người dùng thông qua giọng nói hoặc tín hiệu phím bấm điện thoại. Thông qua IVR, khách hàng có thể tìm thấy câu trả lời cho các thắc mắc phổ biến. Hệ thống sử dụng các đoạn âm thanh được ghi âm sẵn hoặc tạo động để giao tiếp với khách hàng. Trong một tổ chức, hệ thống IVR có thể giải quyết tới 60% lưu lượng cuộc gọi, đặc biệt là đối với các tương tác đơn giản [15].

b) **Flowchart:** Hệ thống IVR có thể được xem là một dạng *flowchart-based conversational system*, trong đó luồng hội thoại được tổ chức theo các *node* và *edge*, tương tự như một **Finite State Machine**. Mỗi bước trong hệ thống tương ứng với một trạng thái cụ thể, trong khi các lựa chọn của người dùng đóng vai trò là các *transition* giữa các trạng thái đó [16].

Flowchart trong IVR thực hiện chức năng điều hướng người dùng thông qua các *menu* phân cấp, đảm bảo rằng người dùng luôn được dẫn dắt đến bước tiếp theo một cách rõ ràng và có kiểm soát. Đồng thời, hệ thống áp dụng các cơ chế *validation* và *loop* (ví dụ: yêu cầu nhập lại khi sai mã PIN) nhằm duy trì tính chính xác và toàn vẹn của dữ liệu.

Ngoài ra, IVR còn có khả năng cá nhân hóa trải nghiệm người dùng ở mức cơ bản, thông qua việc nhận diện số điện thoại gọi đến hoặc liên kết với hệ thống CRM để cung cấp thông tin phù hợp với từng khách hàng. Tuy nhiên, khác với các hệ thống hội thoại dựa trên AI, IVR truyền thống chủ yếu dựa trên *rule-based logic* và không xử lý ngôn ngữ tự nhiên [15].

c) **Kịch bản:** Kịch bản hội thoại trong các hệ thống IVR truyền thống thường được thiết kế theo mô hình có hướng hay còn gọi là mô hình do hệ thống chủ động. Trong kiến trúc này, hệ thống kiểm soát hoàn toàn luồng giao tiếp bằng cách đưa ra các lựa chọn giới hạn, và người dùng buộc phải đối xạ ý định của mình vào các tùy chọn có sẵn tại mỗi trạng thái [17].

Xét kịch bản tương tác với hệ thống hộp thư thoại dưới đây, chúng ta có thể thấy rõ cách một người dùng phải tự dò đường qua một cây quyết định cứng khi ý định ban đầu của họ không nằm ở lớp menu ngoài cùng:

- **Khởi tạo và sự bất đồng bộ ý định:** Tại lượt hội thoại đầu tiên, hệ thống bắt đầu ở *node* gốc và đọc các nhánh có sẵn. Ý định thực sự của người dùng là cài đặt mật khẩu, nhưng do đây là một *sub-node* ẩn ở tầng sâu hơn, người dùng không thể ra lệnh trực tiếp mà buộc phải dùng phương pháp thử-sai bằng cách chọn nhánh *Nghe* [17].
- **Thực thi và vòng lặp tự động:** Khi hệ thống đi vào nhánh *Nghe*, nó thực thi truy xuất dữ liệu (báo không có tin nhắn mới). Ngay lập tức, luồng hội thoại sử dụng một *chuyển tiếp lặp* để ép người dùng quay trở lại trạng thái *Main menu*. Cơ chế này đảm bảo phiên làm việc không bị ngắt quãng sau khi một tác vụ phụ hoàn thành [17].
- **Điều hướng sai và chuyển tiếp toàn cục:** Ở lần thử thứ hai, người dùng chọn nhánh *Tùy chọn trả lời* và đi sâu vào một nhánh phụ. Tuy nhiên, khi nghe các *node* con, họ nhận ra mình đã đi sai luồng. Thay vì mắc kẹt, người dùng phát âm lệnh *Main menu*. Trong thiết kế kịch bản IVR, đây là một lệnh chuyển tiếp toàn cục được gắn vào mọi *node*, cho phép người dùng lập tức hủy bỏ nhánh hiện tại để hội tụ về điểm xuất phát [17].
- **Đạt được mục tiêu:** Cuối cùng, người dùng chọn đúng nhánh *Cài đặt hệ thống*, mở ra các *node* con phù hợp và tìm thấy đích đến là *node* *Mật khẩu* để tiếp tục quy trình nghiệp vụ [17].

Kịch bản này phản ánh rõ nét đặc trưng của mô hình IVR có hướng: Hệ thống cực kỳ ổn định, an toàn và loại bỏ được sự mơ hồ trong dữ liệu đầu vào. Tuy nhiên, do thiếu vắng khả năng xử lý ngôn ngữ tự nhiên để bắt ý định tức thời, luồng kịch bản đòi hỏi người dùng phải trải qua một lộ trình dài, cứng nhắc, phụ thuộc hoàn toàn vào cấu trúc phân cấp mà nhà phát triển đã định nghĩa trước.

d) **Kết quả:** Việc áp dụng *flowchart* trong hệ thống IVR có thể được tóm lược qua ba giai đoạn xử lý chính. Trước hết, hệ thống thực hiện xác thực người dùng bằng cách đối chiếu dữ liệu đầu vào với cơ sở dữ liệu, đồng thời kích hoạt cơ chế lặp có giới hạn nếu thông tin không hợp lệ. Tiếp theo, sau khi xác thực thành công, người dùng được điều hướng tới các nút chức năng trung tâm để lựa chọn dịch vụ thông qua phím bấm hoặc giọng nói, giúp tối giản các bước trung gian. Cuối cùng, hệ thống truy xuất và trả về kết quả kèm theo các cơ chế xử lý lỗi và hoàn tác, cho phép người dùng quay lại các *menu* trước đó, qua đó duy trì luồng hội thoại liên tục và ổn định như một *converging flow* [18].

III. ĐỀ XUẤT HƯỚNG TIẾP CẬN

Nhằm giải quyết triệt để các thách thức trong việc xây dựng một hệ thống hội thoại thông minh nhưng vẫn đảm bảo tính tuân thủ quy trình chặt chẽ, nhóm đề xuất một hướng tiếp cận mang tính kết hợp. Phương pháp này tận dụng sự hiệp đồng giữa cấu trúc logic vững chãi của *flowchart* và khả năng xử

lý ngôn ngữ tự nhiên linh hoạt của các mô hình ngôn ngữ lớn (LLM).

Trọng tâm của kiến trúc này nằm ở việc thiết lập một mô hình vận hành hai lớp: trong đó *flowchart* đóng vai trò như một bộ định tuyến trạng thái điều hướng luồng nghiệp vụ, còn LLM được tích hợp như một agent, một bộ não thực hiện nhiệm vụ giao tiếp, hiểu ngữ cảnh và suy luận rẽ nhánh dựa trên dữ liệu đầu vào từ người dùng.

A. Sự phân tách vai trò giữa Flowchart và LLM

Trong hệ thống đề xuất, LLM không hoạt động theo cơ chế tự do, điều vốn dễ dẫn đến các sai lệch về nghiệp vụ. Thay vào đó, một ranh giới trách nhiệm rõ ràng được thiết lập để tối ưu hóa cả tính kỷ luật lẫn tính tự nhiên:

- **Flowchart** được số hóa và đưa vào hệ thống dưới dạng cấu trúc đồ thị văn bản. Mỗi nút trong đồ thị này đại diện cho một nhiệm vụ nghiệp vụ cụ thể (Ví dụ: *node* SK001 thực hiện nhiệm vụ thu thập thông tin định danh người dùng). Vai trò của *flowchart* là thiết lập các *quy định cứng*, đóng vai trò như một lớp bảo vệ ngăn chặn chatbot bỏ sót các bước bắt buộc hoặc đi chệch khỏi quy trình nghiệp vụ đã được phê duyệt.
- **LLM:** Thay vì sử dụng các cấu trúc rẽ nhánh if-else cứng nhắc và khó bảo trì của lập trình truyền thống, hệ thống sử dụng LLM để đảm nhiệm vai trò trung tâm trong việc đọc hiểu ý định của người dùng. LLM không chỉ đơn thuần sinh câu trả lời tự nhiên dựa trên chỉ thị từ nút hiện tại mà còn đóng vai trò là một bộ thẩm định, tức là nó sẽ phân tích phản hồi của người dùng để đánh giá xem các điều kiện tại các cạnh đã được thỏa mãn hay chưa, từ đó đưa ra quyết định chuyển trạng thái một cách thông minh.

B. Cơ chế xây dựng prompt dựa trên trạng thái

Một trong những vấn đề lớn nhất khi làm việc với LLM là giới hạn về cửa sổ ngữ cảnh và hiện tượng nhiễu thông tin khi nạp quá nhiều dữ liệu. Để khắc phục, nhóm đề xuất phương pháp *cấp phát ngữ cảnh cục bộ* thay vì nạp toàn bộ *flowchart*:

- **Theo dõi trạng thái thời gian thực:** Hệ thống duy trì một bộ theo dõi trạng thái tích cực, luôn ghi nhớ vị trí chính xác của người dùng trong luồng *flowchart* (Ví dụ: *Current_State* = SK001). Điều này giúp thu hẹp phạm vi tìm kiếm logic tại mỗi thời điểm.
- **Cấp phát ngữ cảnh động:** Tại mỗi lượt hội thoại, thay vì đưa toàn bộ tài liệu quy trình, hệ thống chỉ trích xuất thông tin liên quan trực tiếp đến *node* hiện tại và các nhánh rẽ xuất phát từ nó. Điều này giúp LLM tập trung hoàn toàn vào nhiệm vụ hiện tại, giảm thiểu tối đa hiện tượng ảo giác (*hallucination*).
- **Ra quyết định và cập nhật trạng thái:** Dựa trên lịch sử hội thoại gần nhất và prompt được tinh chỉnh theo trạng thái, LLM sẽ thực hiện hai nhiệm vụ song song: sinh ra câu phản hồi thân thiện với người dùng và trả về một tín hiệu điều khiển để hệ thống cập nhật trạng thái mới trên đồ thị.

C. Chiến lược xử lý tình huống ngoài phạm vi flowchart

Thực tế triển khai cho thấy người dùng thường có xu hướng hỏi các câu hỏi bên lề hoặc nhảy cóc quy trình. Để hệ thống không bị đứng máy hoặc phản hồi máy móc, chúng tôi xây dựng cơ chế xử lý linh hoạt:

Hệ thống cho phép LLM sử dụng tri thức nền tảng để giải đáp các thắc mắc phụ của người dùng ngay lập tức. Tuy nhiên, để đảm bảo tính kỷ luật, LLM được lập trình để thực hiện một *lời nhắc khéo* ngay sau câu trả lời, nhằm dẫn dắt người dùng quay trở lại đúng workflow đang dang dở.

Ví dụ, nếu người dùng đang ở bước khai báo thông tin nhưng lại hỏi về giá dịch vụ, LLM sẽ: (1) Nhận diện câu hỏi nằm ngoài điều kiện chuyển tiếp của node hiện tại; (2) Trả lời câu hỏi về giá; (3) Nhắc nhở người dùng quay lại hoàn thành thông tin đang khai báo. Cơ chế này giúp giữ lại trạng thái tại node hiện tại mà không làm gãy đổ luồng hội thoại, tạo ra cảm giác giao tiếp trơn tru như với con người.

D. Cơ chế hậu kiểm

Mặc dù kiến trúc kết hợp mang lại sự linh hoạt lớn, nhưng bản chất xác suất của LLM vẫn tiềm ẩn rủi ro về việc thực thi sai lệch hoặc nhảy bước không hợp lệ. Do đó, một lớp hậu kiểm độc lập được thiết lập như một chốt chặn cuối cùng:

Giai đoạn 1: Duy trì hội thoại và dẫn dắt. Việc tương tác giữa người dùng và chatbot vẫn diễn ra bình thường theo cơ chế đã nêu. Tuy nhiên, khi cuộc hội thoại chạm đến nút kết thúc trên flowchart, LLM sẽ được chỉ thị để giữ chân người dùng trong khi hệ thống thực hiện kiểm tra ngầm.

Giai đoạn 2: Kích hoạt kiểm tra logic độc lập. Tại thời điểm này, một đoạn mã hậu kiểm không phụ thuộc vào LLM sẽ được kích hoạt. Đoạn mã này sẽ truy vấn trực tiếp vào cơ sở dữ liệu và nhật ký hệ thống để rà soát:

- 1) Các trường dữ liệu bắt buộc đã được thu thập đầy đủ và đúng định dạng hay chưa?
- 2) Các API nghiệp vụ cần thiết (ví dụ: kiểm tra số dư, xác thực danh tính) đã được gọi thành công với tham số chuẩn xác hay chưa?

Giai đoạn 3: Tiến hành dựa trên kết quả hậu kiểm. Dựa trên báo cáo từ đoạn mã kiểm tra:

- **Trường hợp thành công:** Hệ thống sẽ xác nhận mục tiêu cuộc hội thoại đã đạt được và thông báo kết quả cuối cùng cho người dùng một cách chính thức.
- **Trường hợp thất bại:** Nếu phát hiện thiếu hụt thông tin hoặc lỗi API, hệ thống sẽ gửi thông báo lỗi cụ thể cho người dùng. Lúc này, chatbot sẽ thực hiện chiến lược điều hướng, đưa người dùng bắt đầu lại quy trình để đảm bảo tính toàn vẹn của dữ liệu.

Để cơ chế này hoạt động hiệu quả, hệ thống yêu cầu một kiến trúc lưu trữ dữ liệu tường minh, nơi mọi trạng thái hội thoại và kết quả gọi API đều được ghi nhận chi tiết, phục vụ cho việc đối soát tức thời.

TÀI LIỆU

- [1] D. Anh-Hoang, V. Tran, and L.-M. Nguyen, "Survey and analysis of hallucinations in large language models: attribution to prompting strategies or model behavior," *Frontiers in Artificial Intelligence*, vol. 8, p. 1622292, 2025.
- [2] M. Karpowicz *et al.*, "On the fundamental impossibility of hallucination control in large language models," *arXiv preprint arXiv:2506.06382*, 2025.
- [3] D. Raghu, S. Agarwal, S. Joshi *et al.*, "End-to-end learning of flowchart grounded task-oriented dialogs," in *Proceedings of the 2021 conference on empirical methods in natural language processing*, 2021, pp. 4348–4366.
- [4] S. Feng, H. Wan, C. Gunasekara, S. Patel, S. Joshi, and L. Lastras, "doc2dial: A goal-oriented document-grounded dialogue dataset," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 8118–8128.
- [5] J. Zou, B. Wang, L. Li, S. Zhang, N. Xu, and J. Zhao, "Floca: Towards faithful and logically consistent flowchart reasoning," *arXiv preprint arXiv:2602.14035*, 2026.
- [6] A. Bordes, Y.-L. Boureau, and J. Weston, "Learning end-to-end goal-oriented dialog," *arXiv preprint arXiv:1605.07683*, 2016.
- [7] P. Budzianowski, T.-H. Wen, B.-H. Tseng, I. Casanueva, S. Ultes, O. Ramadan, and M. Gasic, "Multiwoz-a large-scale multi-domain wizard-of-oz dataset for task-oriented dialogue modelling," in *Proceedings of the 2018 conference on empirical methods in natural language processing*, 2018, pp. 5016–5026.
- [8] M. F. McTear, *The Conversational Interface: Talking to Smart Devices*. Springer, 2016.
- [9] I. Kopeček and L. Škarvada, "Modelling dialogue systems by finite automata," Faculty of Informatics, Masaryk University, Tech. Rep. FIMU-RS-2003-01, 2003.
- [10] M. Serras, M. I. Torres, and A. del Pozo, "Goal-conditioned user modeling for dialogue systems using stochastic bi-automata," in *Proceedings of the 8th International Conference on Pattern Recognition Applications and Methods (ICPRAM)*, 2019, pp. 598–605.
- [11] O. R. Orozko and M. I. Torres, "Online learning of stochastic bi-automaton to model dialogues," in *Proceedings of the 16th Annual SIGdial Meeting on Discourse and Dialogue*, 2015, pp. 432–436.
- [12] D. Raghu, S. Joshi, S. Joshi, and Mausam, "Flonet: A neural framework for flowchart grounded dialog response generation," *arXiv preprint arXiv:2110.09101*, 2021.
- [13] —, "Structural constraints and natural language inference for end-to-end flowchart grounded dialog response generation," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022, pp. 10 763–10 774.
- [14] H. Zhan, S. Maruf, L. Qu, Y. Wang, I. Zukerman, and G. Haffari, "Turning flowchart into dialog: Augmenting flowchart-grounded troubleshooting dialogs via synthetic data generation," in *Proceedings of the 21st Annual Workshop of the Australasian Language Technology Association*, 2023, pp. 128–139.
- [15] M. Thirumaran, S. Soni, and B. G. Gayathry, "An intelligent interactive voice response system for banking domain," in *Proceedings of the 2015 International Conference on Advanced Research in Computer Science Engineering & Technology (ICARCSET 2015)*, 2015, pp. 1–6.
- [16] V. Ramanarayanan, D. Suendermann-Oeft, P. Lange, R. Mundkowsky, A. V. Ivanov, Z. Yu, Y. Qian, and K. Evanini, "Assembling the jigsaw: How multiple open standards are synergistically combined in the halef multimodal dialog system," in *Multimodal interaction with W3C standards: Toward natural user interfaces to everything*. Springer, 2016, pp. 295–310.
- [17] S. Perugini, T. J. Anderson, and W. F. Moroney, "A study of out-of-turn interaction in menu-based, ivr, voicemail systems," in *Proceedings of the SIGCHI conference on Human factors in computing systems*, 2007, pp. 961–970.
- [18] G. Matto, A. Ramadhani, and L. Mjema, "Optimizing navigation strength of ivr systems through the use of a nonchronological pre-programmed prompt approach," 2020.